

MySQL world 샘플 DB

PK · FK · JOIN 관계 수업

city · country · countrylanguage 세 테이블로 관계형 데이터베이스의 핵심을 설명합니다.

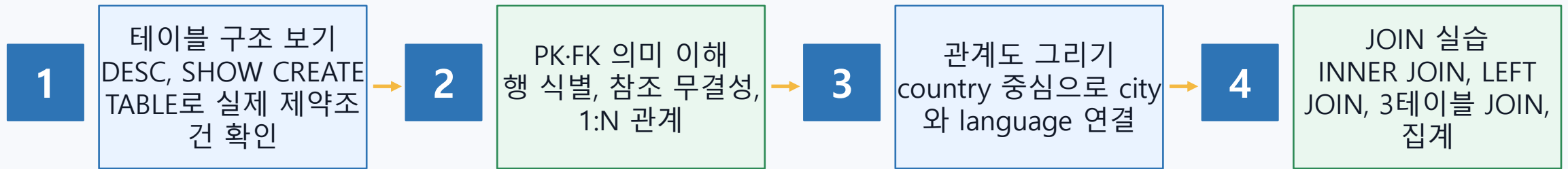
Primary Key

Foreign Key

JOIN

world
샘플 데이터베이스

수업 흐름: 관계를 먼저 보고, JOIN으로 확인하기



- 핵심 질문: "이 컬럼이 어느 테이블의 어떤 컬럼을 가리키는가?"
- JOIN은 관계를 눈으로 확인하는 SELECT 문이다.
- 실습은 예제 실행 → 문제 풀이 → 결과 해석 순서로 진행한다.

world 데이터베이스의 3개 테이블

country
Code PK
Name
Continent
Region
Population
Capital

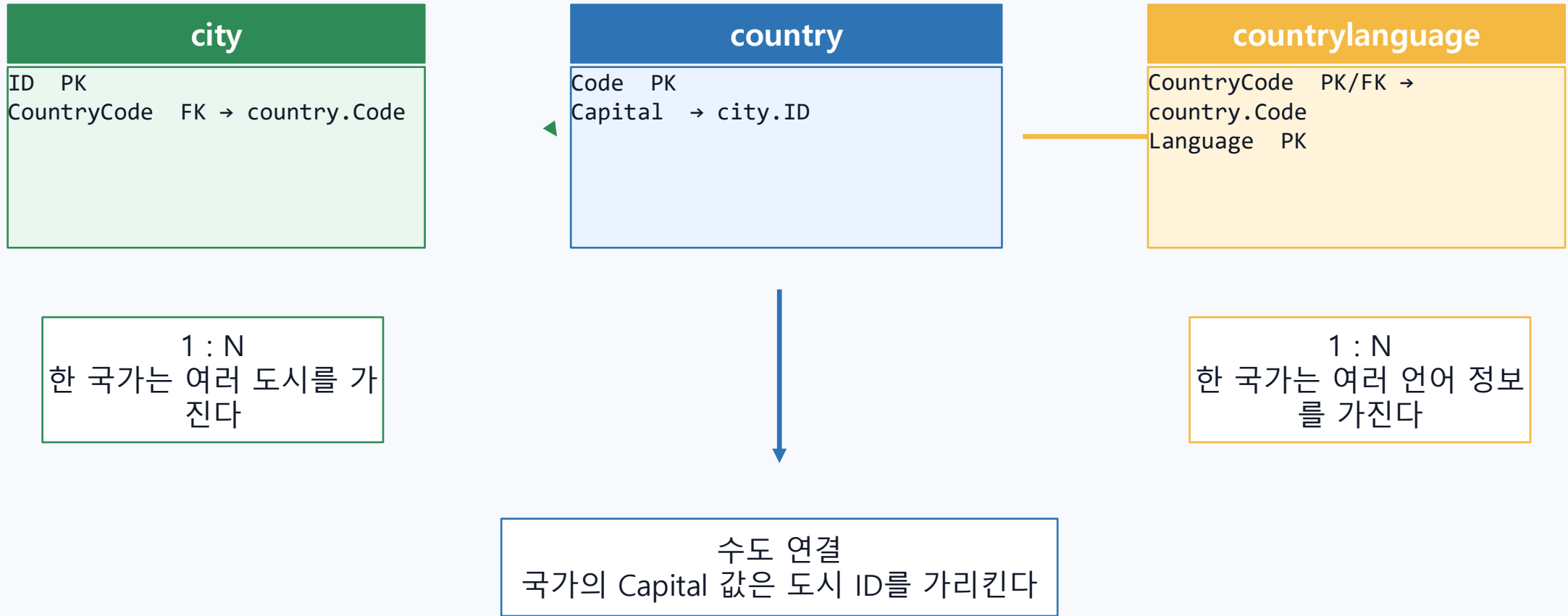
city
ID PK
Name
CountryCode FK
District
Population

countrylanguage
CountryCode PK/FK
Language PK
IsOfficial
Percentage

공식 MySQL 문서 기준: world 샘플 DB는 country, city, countrylanguage 3개 테이블로 구성됩니다.
각 테이블은 국가 정보, 도시 정보, 국가별 언어 정보를 담당합니다.

설치 확인: `USE world; SHOW TABLES;`

관계도: country를 중심으로 연결된다



※ 설치본에 따라 Capital은 실제 FK 제약이거나 단순 인덱스/값 관계일 수 있으므로 SHOW CREATE TABLE로 확인합니다.

Primary Key: 한 행을 구별하는 대표값

country.Code
예: KOR, USA, JPN

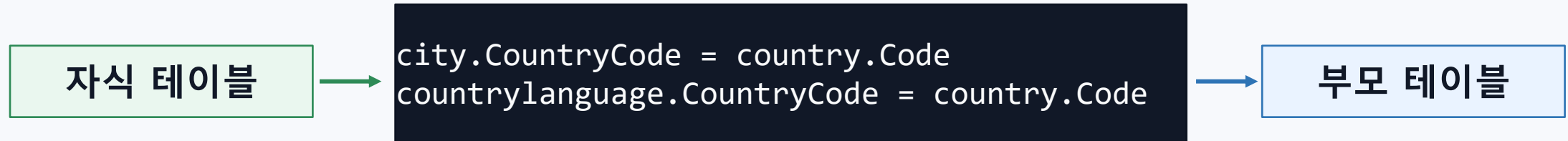
city.ID
도시마다 하나의 번호

countrylanguage
(CountryCode, Language)

- PRIMARY KEY는 중복될 수 없고 NULL이 될 수 없다.
- countrylanguage는 “한 국가 안에서 언어명은 한 번만” 나오면 되므로 복합 기본키를 사용한다.
- 복합 기본키는 두 컬럼을 합쳐서 하나의 행을 구별한다.

```
SHOW KEYS FROM country WHERE Key_name = 'PRIMARY';  
SHOW KEYS FROM city WHERE Key_name = 'PRIMARY';  
SHOW KEYS FROM countrylanguage WHERE Key_name = 'PRIMARY';
```

Foreign Key: 다른 테이블의 행을 가리키는 약속



- FK는 "없는 국가 코드의 도시"가 생기지 않도록 막는다.
- 예: city.CountryCode가 KOR이면 country.Code = KOR인 국가가 있어야 한다.
- 참조 무결성: 연결 대상이 실제로 존재해야 한다는 데이터 품질 규칙이다.

```
SELECT TABLE_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME  
FROM information_schema.KEY_COLUMN_USAGE  
WHERE TABLE_SCHEMA = 'world' AND REFERENCED_TABLE_NAME IS NOT NULL;
```

관계의 종류: 1:N과 수도 연결

country 1 : N city

한 국가에는 도시가 여러 개
도시 한 행은 한 국가에 속함

country 1 : N
countrylanguage

한 국가에는 여러 언어 정보
언어 한 행은 한 국가에 속함

country.Capital → city.ID

국가의 수도 도시를 찾는 연결
LEFT JOIN으로 수도 NULL도 유
지

```
-- 관계를 주로 확인
SELECT CountryCode, COUNT(*) AS city_count
FROM city
GROUP BY CountryCode
ORDER BY city_count DESC
LIMIT 5;
```

```
SELECT CountryCode, COUNT(*) AS language_count
FROM countrylanguage
GROUP BY CountryCode
ORDER BY language_count DESC
LIMIT 5;
```

JOIN의 기본 원리: 연결 조건 ON

JOIN은 “흠어진 정보를 한 결과표로 합치는 SELECT”입니다.



```
SELECT c.Name AS city_name, co.Name AS country_name
FROM city AS c
JOIN country AS co
  ON c.CountryCode = co.Code
WHERE co.Code = 'KOR';
```


INNER JOIN: 연결되는 행만 보여 준다

- city.CountryCode와 country.Code가 같은 행만 결과에 나온다.
- 도시명만 보면 어느 나라 도시인지 헷갈리므로 국가명과 함께 조회한다.
- 별칭 c, co를 쓰면 긴 테이블 이름을 짧게 쓸 수 있다.

```
SELECT
    co.Name AS country_name,
    c.Name AS city_name,
    c.District,
    c.Population AS city_population
FROM country AS co
JOIN city AS c
    ON co.Code = c.CountryCode
WHERE co.Continent = 'Asia'
ORDER BY c.Population DESC
LIMIT 10;
```

읽는 순서

- 1) country와 city를 CountryCode로 연결
- 2) 아시아 국가만 남김
- 3) 도시 인구 내림차순 상위 10개

countrylanguage JOIN: 복합 PK와 언어 정보

```
SELECT
    co.Name AS country_name,
    cl.Language,
    cl.IsOfficial,
    cl.Percentage
FROM country AS co
JOIN countrylanguage AS cl
    ON co.Code = cl.CountryCode
WHERE cl.IsOfficial = 'T'
ORDER BY co.Name, cl.Percentage DESC;
```

복합 기본키

- CountryCode만으로는 한 국가의 여러 언어를 구분할 수 없다.
- Language까지 함께 보아야 한 행이 유일해진다.
- CountryCode는 PK의 일부이면서 FK이기도 하다.

예: KOR + Korean

예: CAN + English

예: CAN + French

3테이블 JOIN: 국가 + 도시 + 언어

```
SELECT
    co.Name AS country_name,
    c.Name AS city_name,
    cl.Language,
    cl.Percentage
FROM country AS co
JOIN city AS c
    ON co.Code = c.CountryCode
JOIN countrylanguage AS cl
    ON co.Code = cl.CountryCode
WHERE co.Code = 'KOR'
ORDER BY c.Population DESC, cl.Percentage DESC;
```

주의: 행이 늘어날 수 있다

- 국가 1개에 도시가 여러 개 있다.
- 국가 1개에 언어 정보도 여러 개 있다.
- 둘을 동시에 붙이면 도시 수 × 언어 수만큼 결과가 늘 수 있다.
- 그래서 3테이블 JOIN은 목적을 분명히 정해야 한다.

LEFT JOIN: 기준 테이블의 행을 모두 남긴다

INNER JOIN

양쪽에 모두 연결되는
행만

LEFT JOIN

왼쪽 테이블 행은 유지
없으면 NULL

```
SELECT
    co.Name AS country_name,
    COUNT(c.ID) AS city_count
FROM country AS co
LEFT JOIN city AS c
    ON co.Code = c.CountryCode
GROUP BY co.Code, co.Name
ORDER BY city_count ASC, co.Name
LIMIT 10;
```

COUNT(c.ID)는 연결된 도시 행만 센다. COUNT(*)와 비교하면 LEFT JOIN에서 차이를 설명하기 좋다.

수도 찾기 JOIN: country.Capital → city.ID

```
SELECT
  co.Name AS country_name,
  cap.Name AS capital_name,
  cap.Population AS capital_population
FROM country AS co
LEFT JOIN city AS cap
  ON co.Capital = cap.ID
WHERE co.Continent = 'Asia'
ORDER BY co.Name;
```

왜 LEFT JOIN?

- Capital이 NULL인 국가도 있을 수 있다.
- 수도 도시 정보가 city에 없을 수도 있다.
- 국가 행을 먼저 보존하고 수도명을 붙이는 목적에 적합하다.

별칭 cap은 city 테이블을 "수도 도시" 역할로 읽기 쉽게 만든 이름입니다.

JOIN + GROUP BY: 관계를 통계로 요약하기

```
SELECT
    co.Continent,
    COUNT(DISTINCT co.Code) AS country_count,
    COUNT(c.ID) AS city_count
FROM country AS co
LEFT JOIN city AS c
    ON co.Code = c.CountryCode
GROUP BY co.Continent
ORDER BY city_count DESC;
```

```
SELECT
    cl.Language,
    COUNT(*) AS country_count
FROM countrylanguage AS cl
GROUP BY cl.Language
HAVING COUNT(*) >= 20
ORDER BY country_count DESC;
```

집계할 때의 질문

- 무엇을 기준으로 묶는가? → GROUP BY
- 묶은 뒤 무엇을 세거나 계산하는가? → COUNT, AVG, MAX
- 묶은 결과를 다시 조건으로 거르는가? → HAVING

실습 운영: 예제 실행 → 문제 풀이 → 설명하기

1. 예제 실행

결과가 어떻게 나오는지 먼저
관찰
ON 조건을 손가락으로 따라가
기

2. 문제 풀이

SELECT 컬럼 → FROM/JOIN →
ON → WHERE
순서로 작성하기

3. 결과 해석

왜 행 수가 늘었는지
왜 NULL이 나오는지 설명하기

```
-- 학생용 파일 구성
01_structure_check.sql  : PK/FK 구조 확인
02_join_examples.sql    : 교사용/학생 실습 예제
03_join_practice.sql    : 학생 풀이 문제
04_join_answers.sql     : 정답 및 해설
```

수업 마무리: 관계형 DB를 읽는 4문장

1. PK는 한 행을 구별하는 대표값이다.

2. FK는 다른 테이블의 PK를 참조해 데이터 오류를 막는다.

3. 1:N 관계에서는 N쪽 테이블에 FK가 들어가는 경우가 많다.

4. JOIN은 PK와 FK 관계를 이용해 흩어진 정보를 한 결과표로 합친다.

다음 단계: ON 조건을 스스로 찾고, INNER JOIN과 LEFT JOIN 결과 차이를 설명할 수 있으면 성공입니다.